

DSC550 Group Project: The Conflicting Stakeholders

Model Selection Under Competing Business Objectives

Deadline for project submission: Friday 27/3 via email

Presentations: Tuesday 31/3 9:00-12:00. Teams will present individually in instructor's office

The Scenario

You are a data science team at **Cyprus Community Bank (CCB)**, a mid-sized retail bank headquartered in Nicosia. CCB offers personal loans ranging from €5,000 to €50,000. The bank processes approximately **10,000 loan applications per year**.

Currently, loan officers manually review each application, a process that is slow, inconsistent, and increasingly unable to keep pace with growing demand. The CEO has asked your team to build a **predictive model** that estimates the probability of loan default and to **recommend a decision threshold** for approving or rejecting applications.

The problem: three senior executives have very different ideas about what “optimal” means.

The Three Stakeholders

1. The CFO: Maria Georgiou (Chief Financial Officer)

Priority: Maximize the bank's net profit from the loan portfolio.

Maria views each loan application as an investment decision: - **Approve a good loan:** The bank earns an average of **€2,400** in interest and fees over the loan lifetime. - **Approve a loan that defaults:** The bank loses an average of **€15,000** (net of partial recovery). - **Reject a good loan:** The bank loses nothing (**€0** net impact). - **Reject a loan that defaults:** The bank avoids the loss (**€0** net impact).

Maria wants the model to **maximize expected profit per application**.

2. The CRO: Andreas Konstantinou (Chief Risk Officer)

Priority: Ensure regulatory compliance and minimize the bank's risk exposure.

The Central Bank of Cyprus has been increasing scrutiny on consumer lending. Andreas is concerned about: - **Approve a good loan:** No risk concern (€0 impact in his framework). - **Approve a loan that defaults:** Regulatory penalties, increased capital requirements, and reputation damage valued at **€45,000 per default**. - **Reject a good loan:** Minor **€500** administrative and reporting cost. - **Reject a loan that defaults:** The correct decision (€0 impact).

Andreas wants the model to **catch as many defaults as possible**, even if it means rejecting some good applicants.

3. The CGO: Elena Papadopoulou (Chief Growth Officer)

Priority: Grow the bank's customer base and market share.

Elena is focused on the long-term value of customer relationships: - **Approve a good loan:** A new customer with a lifetime value (CLV) of **€8,000** (future products, deposits, referrals). - **Approve a loan that defaults:** A loss of **€12,000**, but "the cost of doing business." - **Reject a good loan:** A **€10,000 competitive loss** (the customer goes to a competitor, takes their network with them, and the bank loses market share). - **Reject a loan that defaults:** No cost (€0).

Elena wants the model to **approve as many applicants as possible**, accepting that some will default.

Cost/Benefit Summary Table

Outcome	CFO (Profit)	CRO (Risk)	CGO (Growth)
True Negative (approve good loan)	+€2,400	€0	+€8,000
False Negative (approve default)	-€15,000	-€45,000	-€12,000
False Positive (reject good loan)	€0	-€500	-€10,000
True Positive (reject default)	€0	€0	€0

Note on terminology: In this project, we define the **positive class** as “default” (the event we’re trying to detect). Therefore: - A **True Positive** correctly identifies a defaulter (and we reject them). - A **False Positive** incorrectly flags a good applicant as a defaulter (and we reject them). - A **True Negative** correctly identifies a good applicant (and we approve them). - A **False Negative** misses a defaulter (and we approve them).

Break-Even Thresholds

Each stakeholder’s cost structure implies a different **break-even probability threshold**, the $P(\text{default})$ above which it becomes rational to reject the application.

The formula is:

$$P^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

Where: - C_{FP} = **incremental** cost of a false positive (rejecting a good loan), i.e., the difference between the correct decision and the wrong one: $C_{FP} = \text{TN} - \text{FP}$ - C_{FN} = **incremental** cost of a false negative (approving a default), i.e., the difference between the correct decision and the wrong one: $C_{FN} = \text{TP} - \text{FN}$

For example, the CGO values a new customer at +€8,000 (TN) but suffers a −€10,000 competitive loss from rejecting one (FP), so $C_{FP} = 8,000 - (-10,000) = \text{€}18,000$. The incremental cost captures the full swing between getting it right and getting it wrong.

Your task: Calculate the break-even threshold for each stakeholder using the incremental costs and verify it makes business sense.

The Dataset

The file `cyprus_bank_loans.csv` contains **10,000 historical loan applications** with the following features:

Feature	Type	Description
age	int	Applicant’s age (21-70)
years_employed	int	Years of employment (0-44)

Feature	Type	Description
annual_income	float	Annual income in euros (€15,000-€200,000)
credit_score	int	Credit score (300-850)
debt_to_income_ratio	float	Total debt payments / income (0.01-0.80)
num_existing_loans	int	Number of existing loans (0-8)
previous_defaults	int	Number of previous defaults (0-3)
loan_amount	float	Requested loan amount in euros (€5,000-€50,000)
loan_term_months	int	Loan term (12, 24, 36, 48, or 60 months)
loan_purpose	categorical	Purpose: home_improvement, debt_consolidation, business, personal, education
avg_monthly_balance	float	Average monthly bank balance in euros
months_as_customer	int	Months as a CCB customer (1-360)
default	binary	1 = defaulted, 0 = did not default

Important: The dataset is **imbalanced**: the default rate is approximately 3%. You must account for this in your modeling approach.

Your Deliverables

You will submit **two files** plus a **PDF export**:

1. A Jupyter Notebook (.ipynb) with all code, outputs, and plots
2. A PowerPoint presentation (.pptx) for your presentation
3. A PDF export of the notebook (File → Download as → PDF, or print to PDF)

1. Technical Jupyter Notebook (40% of grade)

Your notebook should be **fully executed** (all cells run, all outputs visible) and include:

1. **Exploratory Data Analysis:** Understand the data, visualize distributions, check for class imbalance
2. **Data Preprocessing:** Handle categorical features, scale if needed, train/test split
3. **Imbalanced Data Handling:** Train models both **with** balancing (`class_weight='balanced'` or SMOTE) and **without** balancing. Compare the two and discuss the tradeoff (see Tip 7 below).
4. **Model Training & Comparison:** Train at least **5 classifiers** from the following list:
 - Logistic Regression
 - Linear Discriminant Analysis (LDA)
 - K-Nearest Neighbors (KNN)
 - Decision Tree
 - Random Forest
 - Gradient Boosting (or XGBoost)
 - Naive Bayes
5. **Evaluation:** For each model, report: Accuracy, Precision, Recall, F1-Score, and AUC. Explain why **accuracy alone is misleading** for this dataset.
6. **Hyperparameter Tuning:** Use GridSearchCV (with cross-validation) on your top 2 models. Justify your choice of scoring metric.
7. **ROC Curves:** Plot all models on one figure. Identify which model gives you the most flexibility for threshold optimization.
8. **Threshold Optimization:** For your best model, find the optimal threshold for **each stakeholder**. Show each operating point on the ROC curve. Calculate the **expected profit per 1,000 applications** for each stakeholder's preferred threshold.
9. **Stakeholder Tradeoff Analysis:** Quantify: *"Satisfying the CRO's requirement costs the CFO €X per 1,000 applications (compared to CFO's optimal threshold)."* Show a complete tradeoff table.

2. Group PowerPoint Presentation + Q&A (60% of grade)

Your presentation consists of two parts:

Presentation (10 minutes): - Deliver your presentation using a PowerPoint file (.pptx) - Cover your approach, key findings, and stakeholder tradeoffs - Include **2–3 slides presenting a decision framework for the CEO** (see below). These slides should be written for a business audience: no code, no jargon. - Every team member must present a section

CEO Decision Framework:

Your job is **not** to tell the CEO which executive to side with. That is a strategic and political decision above your pay grade. Your job is to give the CEO the information needed to make that decision well.

Your framework slides should: - Map out the **feasible range of thresholds** (from the CRO's conservative extreme to the CGO's aggressive extreme), showing what profit, risk exposure, and approval rate look like across that range - Highlight where the **“cheap” compromises** are (e.g., moving slightly from the CFO's optimum toward the CRO's threshold may cost very little profit but substantially reduce regulatory exposure) - Describe what the CEO should **monitor after deployment** (actual default rates, threshold recalibration triggers, sensitivity to changing costs)

The strongest presentations will show the CEO a *range of defensible options with clear tradeoffs*, not a single “right answer.”

Q&A (10 minutes): - Each student must answer at least one question individually - Expect questions on both technical choices and business implications

Tips and Guidance

1. **Start with the cost structures.** Before touching any code, calculate each stakeholder's break-even threshold by hand. This tells you what each stakeholder values and what your model needs to achieve.
2. **Don't chase accuracy.** With a 3% default rate, a model that predicts “no default” for everyone achieves 97% accuracy and catches zero defaults. That's useless.
3. **The ROC curve is your decision-making tool.** Each point on the ROC curve corresponds to a different business decision. The “best” point depends on who you're optimizing for.
4. **There is no single right answer.** The three stakeholders will always disagree. Your job is to quantify the tradeoffs and make a defensible recommendation.
5. **Think about what happens when cost assumptions change.** If regulatory fines doubled, how would your recommendation change? This kind of sensitivity analysis separates good work from great work.
6. **Always sweep thresholds; don't just plug in the break-even formula.** The break-even threshold assumes perfectly calibrated probabilities. In the bank_credit example from class, the theoretical optimum was 0.25 and the empirical optimum was 0.21 (close, but not exact). In this project, the gap can be **much larger** because the dataset is heavily imbalanced (3% vs 27% in the class example). If you use `class_weight='balanced'` or SMOTE to handle the imbalance, the model's predicted probabilities become inflated: a

predicted $P(\text{default}) = 0.15$ might not really mean 15%. The break-even formula still gives you useful intuition about each stakeholder's risk appetite, but the actual optimal threshold must be found empirically by sweeping and calculating profit at each point.

7. **Train both with and without balancing.** Since you are optimizing thresholds anyway (not relying on the default 0.5 cutoff), you don't strictly need `class_weight='balanced'` or SMOTE. A model trained on the imbalanced data will still learn which features predict default; it will just output low probabilities. You then find the right cutoff by sweeping. The advantage: the predicted probabilities stay calibrated, so the break-even formula works more directly. Compare the two approaches: how do the optimal thresholds differ? How does calibration differ? This comparison is a core part of the analysis.

Technical Requirements

- Python 3.8+
- Required libraries: `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `seaborn`
- Optional: `imbalanced-learn` (for SMOTE), `xgboost`
- All code must be reproducible. Use `random_state=42` everywhere: train/test splits, model initializations, SMOTE, `GridSearchCV`. Before submitting, restart your kernel and re-run the entire notebook. If you get different results, you have a reproducibility problem.

Working with AI Tools

You are encouraged to use AI assistants throughout this project. A few tips:

1. **Use a tool that works in your project folder.** Tools like [Claude Code](#), [Codex CLI](#), or [Gemini CLI](#) can read your files, run your code, and iterate with you directly in your working directory. This is far more effective than copying and pasting code into a chat window. An investment for a month's worth of subscription in one of these tools, e.g. ~€20 for Claude Pro for Claude Code (recommended) is well worth it for you to get acquainted with modern AI coding tools.
2. **AI is a collaborator, not a ghostwriter.** Use it to debug, explore ideas, generate boilerplate, or explain concepts. But you must well understand the code you submit and every number you present. The Q&A will test this.

3. **Don't accept the first output.** AI tools often produce plausible but wrong results, especially for domain-specific calculations like cost-based thresholds. Verify the numbers. Check that plots match the data. Read the code critically.
4. **The presentation is yours.** AI can help you draft slides or structure your argument, but the final narrative, the business judgment, and the ability to defend your choices in Q&A must be genuinely yours.

Good luck. The CEO is counting on you.